AI ENGINEERING OS

Technical Specification & Architecture

Version 0.1 — Complete Working Specification

A visual, runtime-first environment where a human CTO/PM directs an Orchestrator / Squad Lead, which coordinates multiple AI engineering harnesses such as Claude Code, Pi, OMP, Freebuff, Codex CLI, Aider, and arbitrary terminal-based agents.

Core principle: The UI is disposable. The runtime is the product.

Prepared as the foundation for implementation / vibe coding.

Document Purpose

This document defines the product model, architectural boundaries, runtime contracts, data model, execution semantics, context system, terminal-harness integration, visual editor, permissions, persistence, observability, milestones, acceptance tests, and implementation guidance for AI Engineering OS.

The specification intentionally prioritizes a reliable headless runtime before the visual canvas. The canvas is a projection and control surface over persistent runtime state; it is not the source of truth.

Table of Contents

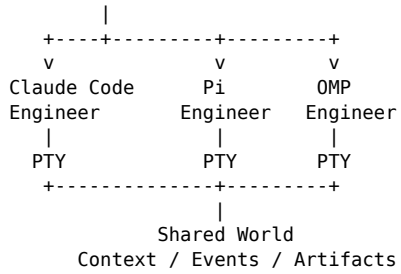
1. Product Definition & Goals

One-liner: A visual workspace where a human CTO/PM directs an Orchestrator agent, which coordinates multiple AI coding/research agents running in their native terminal harnesses.

The product is best understood as an AI Engineering OS: a runtime and workspace for forming a virtual engineering squad out of heterogeneous AI harnesses.

The primary human relationship is hierarchical rather than swarm-based:

```
Human / CTO / PM
        |
        v
Orchestrator / Squad Lead / Tech Lead
```



The user normally gives intent to the Orchestrator. The Orchestrator decomposes work, assigns tasks, monitors execution, resolves failures, requests approvals, and reports outcomes. Users can still directly enter an agent's terminal or chat context when needed.

1.1 Primary Goals

- Coordinate multiple heterogeneous terminal-based AI harnesses in one workspace.
- Allow agents to communicate through a platform-owned structured protocol rather than terminal text.
- Maintain persistent shared state: tasks, artifacts, decisions, events, context references, and agent state.
- Provide a visual node-based representation of execution without making the canvas the source of truth.
- Allow human users to operate primarily through an Orchestrator while retaining direct access to individual workers.
- Support BYOK / model-provider independence and harness-provider independence.
- Make terminal-native agents first-class citizens rather than wrapping them into a proprietary agent implementation.
- Make execution inspectable, replayable, cancellable, resumable, and auditable.

1.2 Success Criterion

The first meaningful product milestone is a complete end-to-end loop:

```
USER:
"Investigate and fix this bug."

    -> ORCHESTRATOR
    -> creates task
    -> assigns Claude Code
    -> Claude runs natively in PTY
    -> produces findings / artifacts
    -> Orchestrator evaluates result
    -> optionally assigns Pi / OMP
    -> tests run
    -> failures route back
    -> fix / verify
    -> Orchestrator reports completion to USER
```

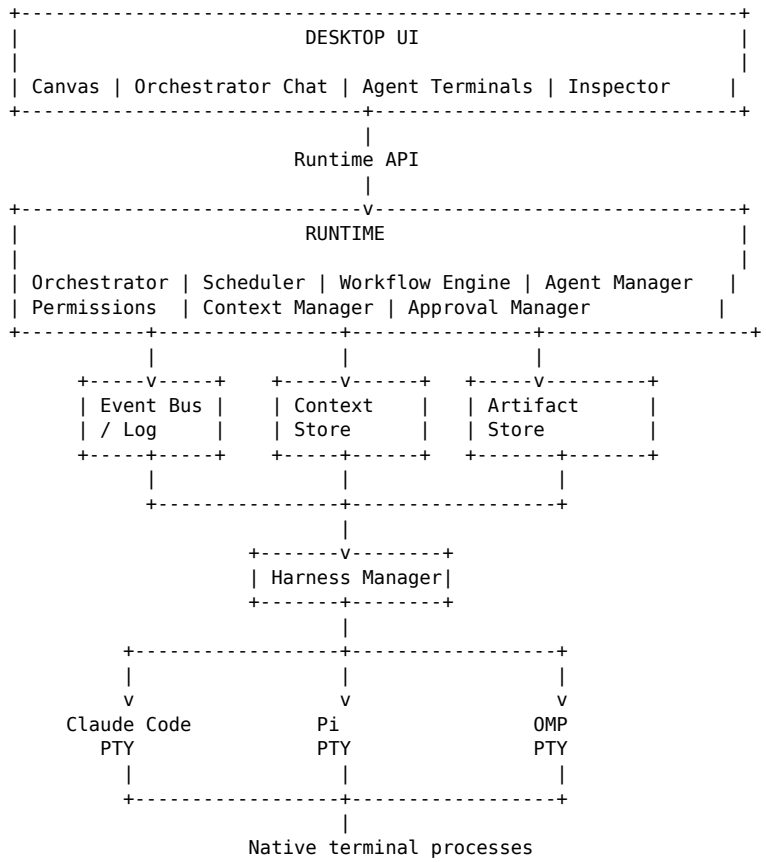
2. Non-Goals & Constraints

- Do not reimplement Claude Code, Pi, OMP, Freebuff, Codex CLI, Aider, or other harnesses.
- Do not make the visual editor the execution engine.
- Do not require every agent to speak a proprietary protocol internally.
- Do not require a distributed infrastructure stack for the initial version.
- Do not assume every harness exposes structured events; the generic PTY adapter must remain viable.
- Do not pass entire conversation histories between agents by default.
- Do not let an LLM directly control core runtime lifecycle, permissions, or scheduling.
- Do not begin with a massive microservice architecture; a local-first monolith/runtime is preferred.

3. Core Concepts

Concept	Definition
Workspace	Top-level persistent environment containing projects, agents, tasks, workflows, events, artifacts, and configura
Project	A concrete code/research/work directory within a workspace.
Orchestrator	Privileged agent acting as Squad Lead / Tech Lead and primary interface for the human.
Agent	Logical worker identity capable of receiving tasks and producing messages/results.
Harness	Runtime adapter that launches and communicates with a concrete agent process or CLI.
Session	One running or historical execution instance of a harness.
PTY	Pseudo-terminal layer used to run interactive terminal-native harnesses.
Task	Unit of work with ownership, status, dependencies, context, and outputs.
Artifact	First-class output/reference such as a file, research document, code change, image, or test result.
Event	Immutable record of an important runtime transition or action.
Message	Structured agent-to-agent or human-to-agent communication envelope.
Context	Selected information supplied to an agent for a particular task/session.
Channel	Logical IRC-like communication stream backed by persistent events.
Workflow	Graph of nodes and edges describing execution/data/control relationships.
Node	Visual/runtime unit such as an agent, tool, condition, approval, or subworkflow.

4. High-Level Architecture



4.1 Architectural Boundaries

- UI layer: rendering, editing, visualization, user input. It may subscribe to runtime state but must not own authoritative state.
- Runtime layer: authoritative execution semantics, lifecycle, scheduling, permissions, task transitions, and persistence.
- Harness layer: process launch, PTY I/O, resize, signals, adapter-specific behavior.
- Context layer: selects and resolves context references into task/session input.
- Event layer: immutable operational history.
- Artifact layer: durable references to outputs and versions.

5. Runtime & Orchestrator

5.1 Orchestrator Responsibilities

- Receive user intent.
- Inspect workspace/project state.
- Decompose intent into tasks.
- Select suitable agents/harnesses.
- Assign tasks and establish dependencies.
- Monitor task and agent state.
- Read relevant events, artifacts, findings, and decisions.
- Request additional work or retries.
- Resolve or escalate conflicts.
- Request human approval for privileged operations.
- Summarize progress and outcomes for the user.

5.2 Deterministic Runtime Rule

LLMs decide; the runtime executes. The Orchestrator may emit a structured decision, but the runtime validates and performs the corresponding transition. No LLM should directly mutate authoritative state outside runtime APIs.

5.3 Orchestrator Loop

```
USER REQUEST
   |
UNDERSTAND
   |
INSPECT STATE
   |
DECOMPOSE
   |
PLAN
   |
ASSIGN
   |
EXECUTE
   |
OBSERVE
   |
EVALUATE
  / retry escalate
  \ /
VERIFY
   |
COMPLETE
   |
REPORT
```

5.4 Suggested Orchestrator Contract

```
interface Orchestrator {
  handleUserMessage(
    workspaceId: string,
    message: string
  ): Promise<OrchestratorResult>

  inspectState(
    workspaceId: string
  ): Promise<WorkspaceSnapshot>

  proposePlan(
    input: OrchestratorInput
  ): Promise<Plan>

  executePlan(
    planId: string
  ): Promise<void>

  handleEvent(
    event: RuntimeEvent
  ): Promise<void>
}
```

6. Harness System & PTY

6.1 Harness Abstraction

```
interface Harness {
  id: string
  type: string
  name: string

  detect(): Promise<boolean>

  spawn(config: SpawnConfig): Promise<HarnessSession>

  send(sessionId: string, input: string): Promise<void>

  resize(sessionId: string, cols: number, rows: number): Promise<void>

  interrupt(sessionId: string): Promise<void>

  terminate(sessionId: string): Promise<void>

  kill(sessionId: string): Promise<void>

  events(sessionId: string): AsyncIterable<HarnessEvent>
}
```

6.2 Harness Types

- GenericTerminalHarness — mandatory P0 adapter; launches arbitrary interactive CLI processes.
- ClaudeCodeHarness — optional specialized adapter when structured integration is beneficial.
- PiHarness — optional specialized adapter.
- OMPHarness — optional specialized adapter.
- FreebuffHarness — optional specialized adapter.
- Future harnesses — Codex CLI, Aider, custom agents, scripts, local models, remote agents.

6.3 Native Terminal Requirement

Harnesses must execute in their real terminal environment. Interactive programs require PTY semantics, including stdin/stdout, terminal resize, signals, and process lifecycle.

6.4 Sideband Communication

Terminal output and platform communication are separate channels:

```
Harness Process
     |
     v
    PTY
     |
     +--> terminal output --> UI
     |
     +--> lifecycle/adapter events --> Runtime
                                        |
                                        +--> Event Bus
                                        +--> Context
                                        +--> Orchestrator
```

7. Agent Communication & Event Bus

7.1 Message Envelope

```
interface AgentMessage {
  id: string
  from: AgentId
  to?: AgentId
  channel?: string
  type:
    | "message" | "task" | "request" | "response"
    | "result" | "question" | "status"
    | "escalation" | "artifact" | "approval"
  payload: unknown
  replyTo?: string
  contextRefs?: ContextReference[]
  timestamp: number
}
```

7.2 Event Envelope

```
interface RuntimeEvent {
  id: string
  workspaceId: string
  timestamp: number
  source: EntityId
  type: string
  payload: unknown
}
```

7.3 IRC-like Channels

Channels provide human-readable communication rooms while remaining backed by an event stream. Examples: #general, #architecture, #frontend, #backend, #testing, #debug, #research, #review.

7.4 Important Rule

The channel is not the system of record. Events are. The channel is a projection/filter over persistent messages and events.

8. Context System

8.1 Context Hierarchy

```
GLOBAL
  |
WORKSPACE
  |
PROJECT
  |
WORKFLOW
  |
TASK
  |
```

```
AGENT
  |
SESSION
```

8.2 Context Injection Principle

Agents should receive the smallest useful context set, not the entire project history. Context should be assembled from references to tasks, artifacts, decisions, files, messages, and selected events.

8.3 Context Reference

```
interface ContextReference {
  type:
    | "artifact" | "event" | "message"
    | "task" | "decision" | "file"
  id: string
  relevance?: number
}
```

8.4 Context Adapter

Different harnesses may have different context injection mechanisms. The Context Adapter translates platform context into the harness's available interface: prompt, file, stdin, environment, CLI argument, structured API, or other supported mechanism.

9. Tasks, Workflows & Scheduling

9.1 Task Model

```
interface Task {
  id: string
  title: string
  description: string
  status:
    | "pending" | "assigned" | "running"
    | "blocked" | "completed" | "failed" | "cancelled"
  assignedTo?: AgentId
  parentTaskId?: string
  dependencies: string[]
  contextRefs: ContextReference[]
  priority: number
}
```

9.2 Workflow Graph

A workflow is a serializable graph. It should support deterministic control flow while allowing agentic nodes to make bounded decisions.

9.3 Edge Types

- Data edge — transfers or references a data output.
- Control edge — expresses execution dependency.
- Conditional edge — follows a runtime-evaluated condition.
- Error edge — receives failures.
- Approval edge — waits for human authorization.

9.4 Scheduler Requirements

- Only runnable tasks may start.
- Dependencies must be satisfied before execution.
- Concurrency limits must be enforced.

- Cancellation must propagate according to policy.
- Failed tasks may retry according to explicit policy.
- The scheduler must persist state before/after important transitions.
- The scheduler must tolerate process crashes and resume from durable state.

10. Artifacts & Workspace State

10.1 Artifact Model

```
interface Artifact {
  id: string
  type:
    | "file" | "document" | "code"
    | "image" | "research" | "result"
  name: string
  uri: string
  version: number
  createdBy: EntityId
  metadata: Record<string, unknown>
}
```

10.2 Artifact Principles

- Prefer references over copying large content into messages.
- Artifacts should have provenance: who/what created them and when.
- Versions should be inspectable.
- Code artifacts should map to real filesystem paths where applicable.
- Generated reports/results should remain available after agent termination.

10.3 Decisions & Project Memory

The workspace should retain durable decisions, requirements, constraints, architecture notes, open issues, and lessons learned. The Orchestrator is the primary consumer and maintainer of this project-level memory.

11. Permissions & Human Approval

11.1 Capability Model

```
interface AgentPermissions {
  filesystem: boolean
  terminal: boolean
  network: boolean
  browser: boolean
  git: boolean
  github: boolean

  spawnAgents: boolean
  assignTasks: boolean
  deploy: boolean
}
```

11.2 Suggested Defaults

Capability	Engineer	Orchestrator	Human
Filesystem	Allowed within project	Allowed	Allowed
Terminal	Allowed	Allowed	Allowed
Git	Allowed	Allowed	Allowed

Capability	Engineer	Orchestrator	Human
Spawn agents	Denied	Allowed	N/A
Assign tasks	Denied	Allowed	Allowed
Production deploy	Approval	Approval	Allowed
Destructive operations	Approval/policy	Approval/policy	Allowed

11.3 Human Approval Flow

```
Agent
  |
approval.requested
  |
Orchestrator
  |
Human
  +--> approve --> resume
  |
  +--> deny ------> stop/replan
```

12. Visual Canvas & Node Model

12.1 Canvas Principle

The canvas is a visualization and control surface over the workflow definition and runtime state. It must not be the authoritative source of execution state.

12.2 Node Categories

Category	Examples
Agent	Claude Code, Pi, OMP, Generic Agent
Tool	Terminal, Browser, GitHub, Filesystem
Control	Condition, Router, Merge, Split, Loop, Approval
Workflow	Subworkflow / Subgraph
Human	Input, Approval, Review

12.3 Serializable Workflow Definition

```
{
  "version": 1,
  "nodes": [
    {
      "id": "researcher",
      "type": "agent.researcher",
      "position": { "x": 100, "y": 200 },
      "config": { "model": "..." }
    }
  ],
  "edges": [
    {
      "source": "researcher",
      "target": "architect",
      "kind": "data"
    }
  ]
}
```

12.4 Visual Technology

Use XYFlow / React Flow initially for node editor primitives. Keep the graph model independent of the UI library so it can be replaced later.

13. User Experience & Interaction Model

13.1 Primary Interaction

The default conversation is with the Orchestrator. The user speaks in intent, not low-level workflow instructions.

```
USER:
"Fix the combat formation issue."

ORCHESTRATOR:
"I'll inspect the formation system, assign an
implementation agent, and run verification."

Canvas:
Claude Code -> Pi -> OMP -> Claude Code

User sees:
- current plan
- squad state
- progress
- blockers
- important decisions
- final result
```

13.2 Direct Worker Interaction

Every worker should be directly inspectable. The user may open its terminal, send a message, inspect context, inspect events, review artifacts, or change its task. Such interventions become runtime events and remain visible to the Orchestrator.

13.3 Squad State

The primary dashboard should show active agents, task progress, blocked work, failures, approvals, token/cost metrics, and the latest meaningful event.

14. MCP, Browser, Git & External Tools

MCP should be treated as a capability/tool integration layer, not the orchestration protocol. The runtime owns agent communication, tasks, events, context, permissions, scheduling, and workflow semantics.

14.1 Tool Integration Categories

- MCP servers — external capabilities such as GitHub, filesystem, browser, databases, monitoring, and other services.
- Browser — Playwright-backed browser sessions represented as tool or browser nodes.
- Git — local repository operations, branches, commits, diffs, status, and later remote provider integrations.
- Filesystem — project-scoped access.
- Terminal — deterministic command execution independent of an agent harness.

14.2 Browser Node

A browser session should be independently inspectable and attachable to tasks. Browser findings may become artifacts and context references.

15. Persistence & Data Model

15.1 Initial Storage

Use SQLite for V0/V1. Use Drizzle ORM for schema and migrations. Avoid introducing Postgres, Redis, Kafka, NATS, or other distributed components until actual scale requirements justify them.

Table	Purpose
workspaces	Workspace metadata and settings
projects	Project/repository roots
nodes	Workflow/node definitions
edges	Graph relationships
agents	Logical agent identities
harnesses	Installed/registered harness definitions
sessions	Running/historical process sessions
tasks	Task state and ownership
messages	Structured communication
events	Immutable event log
artifacts	Artifact metadata and references
workflows	Serialized workflow definitions
approvals	Human approval requests and decisions
decisions	Durable project decisions

15.2 Event-Sourced Direction

V0 can use conventional tables plus an append-only event table. The architecture should preserve the option to rebuild projections from events later.

16. Observability, Replay & Debugging

Every important state transition should be observable. The system should answer: What happened? Which agent did it? What context did it receive? What tools ran? What artifacts changed? Why did the Orchestrator choose the next step?

16.1 Node Inspector

```
AGENT: Claude Code
Status: RUNNING
Task: Fix formation separation
Duration: 12m 42s
Tokens: 14,821
Cost: $0.031

Context:
  6 artifacts
  12 events
  4 decisions

Last event:
  tool.completed

Terminal:
  $ claude
  > inspecting FormationSystem.ts
```

16.2 Replay

The runtime should be able to reconstruct a task/workflow's state from persisted events and durable records. Full deterministic replay of LLM outputs is not required; replay should mean reconstructing runtime state and execution history.

17. Reliability & Failure Handling

17.1 Required Failure Classes

- Harness process crash.
- PTY disconnect.
- Agent timeout.
- Tool failure.
- Network/provider failure.
- Malformed agent decision.
- Task dependency failure.
- Context resolution failure.
- Artifact write failure.
- Runtime restart.

17.2 Recovery Principles

- Persist important state transitions before assuming success.
- Treat process death as an observable state, not an exceptional impossibility.
- Support cancellation and explicit termination.
- Retries must be bounded and policy-driven.
- A failed worker should not corrupt the whole workspace.
- The Orchestrator should be able to replan after failures.
- Runtime restart should recover durable tasks/sessions where possible.

18. Security

- Never expose raw API keys to worker agents unless explicitly configured.
- Prefer scoped credentials and environment isolation.
- Project filesystem access should be scoped.
- Destructive and production operations should be approval-gated.
- External tool permissions should be explicit.
- Audit important privileged actions as events.
- Treat agent-generated commands as untrusted input to policy enforcement.
- Do not allow an agent to silently elevate its own permissions.

19. Technology Stack

Layer	Initial Choice	Reason
UI	React + TypeScript + Vite	Existing skillset and fast iteration
Canvas	XYFlow / React Flow	Mature node-editor primitives
State	Zustand or runtime query cache	UI state only; runtime remains authoritative

Layer	Initial Choice	Reason
Runtime	TypeScript/Node initially	Fast vibe-coding and ecosystem access
ORM	Drizzle	Typed SQLite schema/migrations
DB	SQLite	Local-first, simple, durable
PTY	Native PTY layer / library	Interactive terminal compatibility
Desktop	Tauri candidate	Strong native/runtime story
Browser	Playwright	Browser automation/session support
Tools	MCP	External capability integration
Git	Native git CLI initially	Predictable repository behavior
Monorepo	pnpm + Turborepo	Simple package boundaries

Important: the PTY/runtime abstraction must remain isolated enough that the desktop shell can change later. A Node/Electron prototype is acceptable if it accelerates the first reliable milestone.

20. Repository Structure

```
ai-engineering-os/
├── apps/
│   ├── desktop/
│   └── web/
├── packages/
│   ├── core/
│   ├── runtime/
│   ├── orchestrator/
│   ├── context/
│   ├── events/
│   ├── artifacts/
│   ├── tasks/
│   ├── harness/
│   ├── harness-generic/
│   ├── harness-claude/
│   ├── harness-pi/
│   ├── harness-omp/
│   └── ui/
├── docs/
└── examples/
```

Package boundaries are logical, not a mandate to create a complex microservice system. Early implementation can keep several packages in one process.

21. Implementation Milestones

Priority	Milestone	Definition of Done
P0	Headless runtime	Workspace, tasks, event bus, persistence, generic PTY harness, one real agent.
P0	Orchestrator loop	User request -> plan -> assignment -> observation -> result.
P0	Agent communication	Structured worker-to-worker messages through runtime.
P0	Artifact references	Workers can produce and consume durable artifacts.
P1	Multi-agent	Multiple simultaneous harness sessions with dependencies/retries.
P1	Harness adapters	Pi/OMP/Freebuff adapters or robust generic configuration.
P1	Direct worker interaction	User can inspect and message individual agents.
P2	Visual canvas	Live graph projection using XYFlow.
P2	Terminal nodes	Native interactive terminals embedded in canvas.
P2	Context inspector	Show context refs, artifacts, events, task state.
P3	MCP	Tool capability integration.

Priority	Milestone	Definition of Done
P3	Approvals & permissions	Policy and human approval gates.
P3	Replay & recovery	Runtime reconstruction and restart recovery.
P4	Hierarchical squads	Tech leads / QA leads / research leads.

22. Acceptance Tests

22.1 P0 Golden Path

```
1. User creates workspace.
2. User selects project directory.
3. User sends:
   "Investigate this bug."
4. Orchestrator creates a task.
5. Orchestrator launches a generic terminal harness.
6. Harness starts a real interactive CLI agent.
7. Agent operates through PTY.
8. Agent emits observable runtime events.
9. Agent creates an artifact/finding.
10. Orchestrator consumes the result.
11. Orchestrator reports to the user.
12. Workspace is closed and reopened.
13. Task/event/artifact history remains available.
```

22.2 Multi-Agent Acceptance

```
User:
"Fix the failing combat test."

Orchestrator:
  -> Claude Code: investigate
  -> Pi: independently propose fix
  -> OMP: run tests

Claude and Pi communicate through runtime messages.
OMP consumes the selected implementation artifact.
Test failure routes back to Orchestrator.
Orchestrator reassigns follow-up work.
User receives a concise final report.
```

22.3 Direct Intervention Acceptance

```
User opens Claude Code node.
User sends direct instruction.
Instruction becomes a runtime event.
Claude receives it.
Orchestrator sees the intervention.
Task/context state remains coherent.
```

22.4 Failure Acceptance

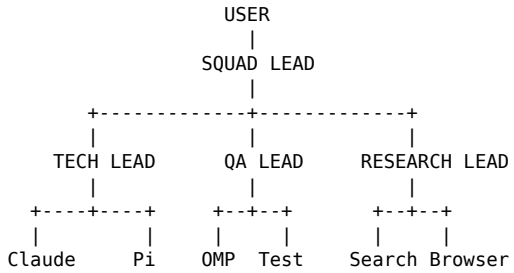
```
Worker crashes.
Runtime marks session CRASHED.
Task becomes BLOCKED or RETRYABLE.
Orchestrator receives event.
Orchestrator chooses:
  - restart
  - reassign
  - escalate
Workspace remains healthy.
```

23. Future / Advanced Capabilities

- Hierarchical agent organizations: Squad Lead -> Tech Lead -> QA Lead -> engineers.
- Agent capability discovery and automatic role assignment.
- Context optimization / relevance ranking.

- Long-term project memory.
- Workflow templates and reusable subgraphs.
- Agent pools and concurrency quotas.
- Remote workers.
- Containerized sandbox execution.
- Cost-aware scheduling.
- Model routing by task difficulty.
- Local-only orchestration for privacy-sensitive projects.
- Shared browser sessions.
- Visual diffing and artifact lineage.
- Human-in-the-loop checkpoints.
- Team collaboration and workspace sharing.
- Plugin ecosystem for harness adapters.
- Remote execution agents.
- Agent performance analytics.
- Automatic postmortems / lessons learned.

23.1 Hierarchical Organization Example



24. Glossary

Term	Meaning
AI Engineering OS	Working product name for the complete runtime + workspace.
Orchestrator	Privileged Squad Lead / Tech Lead agent that coordinates workers.
Harness	Adapter/runtime wrapper around a concrete terminal or agent process.
PTY	Pseudo-terminal providing interactive terminal semantics.
Worker	Non-orchestrator agent performing assigned tasks.
Sideband	Platform communication channel separate from terminal stdout/stderr.
Context Bus	Conceptual layer for selecting and distributing shared context.
Event Bus	Runtime event transport backed by durable event history.
Artifact	Persistent, referenceable output or input object.
Channel	IRC-like logical stream over messages/events.
Control Edge	Workflow dependency describing when execution may proceed.
Data Edge	Workflow connection describing data/reference transfer.
Projection	UI representation of authoritative runtime state.
Golden Path	Minimal end-to-end workflow that proves the system works.

Final Architecture Checklist

- Human primarily communicates with Orchestrator.
- Orchestrator is a privileged runtime agent, not merely a chat window.
- Workers can be Claude Code, Pi, OMP, Freebuff, Codex CLI, Aider, or arbitrary terminal programs.
- Generic PTY support is P0.
- Terminal I/O is separate from structured agent communication.
- Agents communicate through messages/events and can subscribe to IRC-like channels.
- Context is reference-based and selectively injected.
- Artifacts are first-class and durable.
- Tasks have ownership, dependencies, lifecycle, and priorities.
- Runtime owns scheduling, lifecycle, permissions, cancellation, retries, and persistence.
- Canvas is a projection/control surface, never the source of truth.
- SQLite is sufficient for the initial local-first runtime.
- MCP is a capability protocol, not the orchestration protocol.
- Human approvals gate sensitive actions.
- Everything important is observable through events.
- The first demo must prove a complete multi-agent fix/verify loop.

Core Design Mantra

Human intent → Orchestrator → Tasks → Harnesses → Shared Context → Artifacts → Verification → Human outcome.

The UI is disposable. The runtime is the product.

Harnesses are replaceable. The protocol is the product.

LLMs are replaceable. State is not.